Jyothi Engineering College (AUTONOMOUS)
Reaccredited with NAAC (Grade A) and NBA Programmes (CE, CS, EC, EE, ME, MR)
Jyothi Hills, P. O. Vettikkattiri, Cheruthuruthy
Thrissur, Kerala, India, 679531
04884 259000 | info@jecc.ac.in | www.jecc.ac.in
Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University
A Centre of Excellence in Science and Technology by the Catholic Archdiocese of Trichur

PROJECT REPORT

FIRE FIGHTING ROBOT

PBCST504 - MICROCONTROLLERS

Submitted By

MEGHA YELDO	**JEC24CS129**
M MALAVIKA	**JEC24CS133**
NEHA JIMMY	**JEC24CS146**
NIDHI RAJESH	**JEC24CS149**
NIRANJANA K G	**JEC24CS152**
PRARTHANA RAJESH	**JEC24CS158**
SARAYU DINESH P	**JEC24CS168**

To

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of the Degree

of

Bachelor Of Technology (B. Tech)

in

COMPUTER SCIENCE AND ENGINEERING

(AUTONOMOUS)
CREATING TECHNOLOGY LEADERS OF TOMORROW
ESTD 2002

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We hereby declare that the project report entitled **"FIRE FIGHTING ROBOT"**, submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering of APJ Abdul Kalam Technological University, is a bonafide record of the work done by us under the guidance and supervision of our project guide.

We further declare that this work has not been submitted to any other University or institution for the award of any degree, diploma, or other similar titles.

Name of Students		**Signature**
MEGHA YELDO	JEC24CS129	
M MALAVIKA	JEC24CS133	
NEHA JIMMY	JEC24CS146	
NIDHI RAJESH	JEC24CS149	
NIRANJANA K G	JEC24CS152	
PRARTHANA RAJESH	JEC24CS158	
SARAYU DINESH P	JEC24CS168	

Place:

Date:

VISION OF THE DEPARTMENT

Creating ethical leaders in the domain of Computational Sciences through quality professional education with a focus on holistic learning and excellence.

MISSION OF THE DEPARTMENT

- To create technically competent and ethically conscious graduates in the field of Computer Science and Engineering by encouraging holistic learning and excellence.
- To prepare students for careers in Industry, Academia and the Government.
- To instill Entrepreneurial Orientation and research motivation among the students of the department.
- To emerge as a leader in education in the region by encouraging teaching, learning, industry and societal connect.

PROGRAMME EDUCATIONAL OBJECTIVES

1. **PEO1:** The graduates shall have sound knowledge of Mathematics, Science, Engineering and Management to be able to offer practical software and hardware solutions for the problems of industry and society at large.

2. **PEO2:** The graduates shall be able to establish themselves as practicing professionals, researchers or Entrepreneurs in computer science or allied areas and shall also be able to pursue higher education in reputed institutes.

3. **PEO3:** The graduates shall be able to communicate effectively and work in multidisciplinary teams with team spirit demonstrating value driven and ethical leadership.

PROGRAMME SPECIFIC OUTCOMES

1. **PSO1:** An ability to apply knowledge of data structures and algorithms appropriate to computational problems.

2. **PSO2:** An ability to apply knowledge of operating systems, programming languages, data management, or networking principles to computational assignments.

3. **PSO3:** An ability to apply design, development, maintenance or evaluation of software engineering principles in the construction of computer and software systems of varying complexity and quality.

PROGRAMME OUTCOMES

1. **PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop solutions to complex engineering problems.

2. **PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

3. **PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for public health, safety, cost, and environment.

4. **PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data.

5. **PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

6. **PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, and legal framework.

7. **PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

8. **PO8: Individual and Collaborative Teamwork:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

9. **PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as writing effective reports and design documentation.

10. **PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making to one's own work and to manage projects in multidisciplinary environments.

11. **PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for independent and life-long learning, adaptability to new technologies, and critical thinking.

SDGS ADDRESSED BY THE MICRO PROJECT AND JUSTIFICATION

Project Title	**SDG**	**Justification**
Fire Fighting Robot	SDG 7: Affordable and Clean Energy	Promotes efficient energy usage and safety through automated monitoring.
	SDG 11: Sustainable Cities and Communities	Supports smart infrastructure and enhances safety against fire hazards.
	SDG 12: Responsible Consumption and Production	Minimizes resource wastage and ensures prompt response mechanisms.
	SDG 13: Climate Action	Helps mitigate environmental damage caused by uncontrolled fire hazards.

ABSTRACT

Fire incidents can threaten people, property, and the environment. This mini project presents the design of an Arduino-based Fire Fighting Robot intended to demonstrate flame sensing, basic directional movement, water-pump actuation, gas detection, and GSM-based notification. The prototype uses an Arduino Uno R3 as the controller. Left, right, and forward flame-sensor modules provide digital inputs for the movement logic. A geared-motor drive with an L293D motor driver moves the robot. A micro servo mounted near the water outlet sweeps the nozzle, while a 5 V mini water pump supplies the demonstration spray through a suitable switching interface.

An MQ-2 gas sensor is connected to a digital input to indicate a gas/smoke-related trigger. A SIM800L GSM module communicates with the Arduino through SoftwareSerial and uses AT commands to send an SMS and place a phone call. The supplied program configures the sensor pins, motor outputs, pump output, servo, and GSM module. When the forward flame sensor is triggered, the program sets a fire flag and enters the fireresponse routine. When the gas input is active, it attempts to send an SMS.

The project combines embedded programming, sensor interfacing, motor control, actuator switching, and cellular communication in one educational system. Its practical performance depends on sensor calibration, correct motor polarity, power-supply stability, GSM network availability, pump flow, and the precise logic levels of the sensor modules. The supplied code should be tested and refined before demonstration; the report therefore distinguishes intended operation from results that must be measured on the actual prototype. The robot is a learning prototype and not a certified fire-safety device.

TABLE OF CONTENTS

Vision & Mission of Institute and Department **2**

Programme Educational Objectives (PEOs) & PSOs **3**

Programme Outcomes (POs) **4**

SDGs Addressed by the micro project and Justification **5**

Abstract **6**

1. Introduction **1**

2. Objectives **3**

2.1 Specific Objectives 3

2.2 Success Criteria 3

3. Tools Used **4**

3.1 Hardware Components 4

3.2 Software 5

3.3 Communication and Interfaces 5

3.4 Important Power Note 5

4. System Architecture **6**

4.1 Functional Block Diagram 6

4.2 Architecture Description 6

4.3 Component-Level Architecture 7

4.4 Pin Assignment from the Supplied Code 8

5. Methodology **9**

5.1 Overall Working Sequence 9

5.2 Control Logic 9

5.3 Simplified Decision Table 10

5.4 Fire Response Routine 10

6. Implementation **11**

6.1 Mechanical Assembly 11

6.2 Electrical Assembly 11

6.3 Pre-Power Checklist 11

6.4 Program Structure 12

6.5 Setup Function 12
6.6 Main Loop 12
6.7 Function Summary 12
6.8 Fire Response and Pump Operation 13
6.9 SMS Procedure 13
6.10 Call Procedure 13
6.11 Phone Number and Privacy 13
6.12 Calibration 14
6.13 Safety Measures 14
6.14 Code Limitations to Review 14
6.15 Prototype Hardware and Testing 15

7. Results and Discussion **16**

8. Source Code **17**

9. Conclusion **22**

10. References **23**

INTRODUCTION

Background

Fire safety is an important concern in homes, laboratories, workshops, warehouses, and industrial environments. In the early stage of a fire, timely detection and communication can help people respond appropriately. However, approaching a fire may expose people to heat, smoke, toxic gases, or structural hazards. Robotics and embedded systems provide a useful learning platform for exploring remote sensing and actuation.

A mobile robot can use sensors to detect a condition, a microcontroller to make a decision, and actuators to perform a response. In this project, flame-sensor modules provide directional clues, motors provide mobility, a pump provides a small water spray, and a GSM module provides remote notification. The design is intentionally based on commonly available educational components.

Embedded System Concept

An embedded system combines hardware and software to perform a defined task. The Arduino Uno R3 reads sensor inputs and controls outputs according to a program. Sensor signals are inputs; motor-driver control pins, the pump-switching signal, the servo signal, and GSM serial commands are outputs. The main loop repeatedly checks sensor states and selects a response.

Scope

The scope is a small educational prototype operating in a controlled demonstration area. It illustrates flame-direction detection, basic movement, a servo sweep, pump switching, and a gas-triggered SMS. It does not include certified fire classification, obstacle mapping, autonomous navigation, temperature feedback, or a guaranteed extinguishing capability.

Problem Statement

A person may need to approach a potential fire source to inspect it or operate a response device. The project explores whether a small mobile platform can demonstrate a limited remote response using low-cost sensors and actuators. The central challenge is to detect a flame reliably, move in a useful direction, operate the pump at the right time, and communicate an alert.

Proposed Solution

The proposed system uses three digital flame-sensor inputs placed at the left, right, and front of the robot. The controller uses their logic states to select forward movement, a left/right turning action, or a stop state. A fire flag activates a routine that stops the drive outputs, turns on the pump output, sweeps the servo from approximately 50° to 110°, and then turns the pump output off. The code also checks an MQ-2 digital output and sends an SMS when its input reads LOW. During the fire-response routine, it places a call to a configured phone number.

Design Considerations

Sensor modules may output either active-LOW or active-HIGH signals depending on their design and adjustment. The code assumes active-LOW flame and gas triggers. The motor-driver inputs and pump interface must be wired to match the program. The SIM800L requires a suitable supply capable of handling current peaks; it should not be powered directly from the Arduino 5 V pin. These conditions must be verified during assembly.

Report Organisation

The following sections describe the objectives, tools, system architecture, methodology, implementation, expected results, analysis, conclusion, and references. Hardware-specific values and institutional information should be updated to match the actual prototype and department format.

OBJECTIVES

The main objective is to design and demonstrate a small Arduino-controlled mobile robot that combines flame sensing, movement, water-pump operation, gas detection, and GSM notification.

2.1 Specific Objectives

1. Interface left, right, and forward flame-sensor modules with an Arduino Uno R3.
2. Implement a basic movement routine that reacts to the digital states of the flame sensors.
3. Control two geared DC motors through an L293D motor-driver arrangement.
4. Operate a 5 V mini water pump through a suitable relay/driver interface.
5. Use a micro servo to sweep the water outlet across a limited angle.
6. Read the digital output of an MQ-2 gas sensor and trigger a text alert when the input is active.
7. Use a SIM800L GSM module to send an SMS and make a call through AT commands.
8. Assemble the components on a mobile chassis with four wheels and organise the wiring.
9. Test the modules individually before conducting an integrated demonstration.
10. Identify technical limitations and propose improvements to reliability and safety.

2.2 Success Criteria

The prototype may be considered functionally demonstrated when the sensor inputs can be observed, motor commands produce the intended movement, the servo sweeps without binding, the pump interface switches reliably, and the GSM module successfully sends the intended message or call under suitable network conditions. Record each criterion as pass/fail based on actual tests; do not mark it passed without evidence.

TOOLS USED

3.1 Hardware Components

The prototype is built using readily available educational hardware components. The table below outlines each component and its specific purpose in the prototype:

Component	Purpose in the Prototype
Arduino Uno R3	Reads digital sensor inputs and runs the control program.
IR/flame sensor modules (left, right, front)	Provide digital signals used for basic flame-direction decisions.
MQ-2 gas sensor module	Provides a gas/smoke-related sensor output; requires warm-up and calibration.
SIM800L GSM module	Sends SMS and initiates a phone call using AT commands.
L293D motor driver	Interfaces the Arduino control signals with the DC geared motors.
Geared DC motors and 4 wheels	Provide movement of the robot chassis.
5 V mini water pump	Produces a small water spray for a controlled demonstration.
5 V relay module / suitable driver	Switches the pump load; exact wiring depends on the module and pump current.
Micro servo motor	Moves the nozzle or outlet through a limited sweep.
Rechargeable battery	Provides portable power; voltage and current ratings must match loads.
DC buck converter	Regulates battery voltage to a suitable level for compatible modules.
Mini breadboard and jumper wires	Support low-current prototyping connections.

Additional tools normally required: USB cable, computer, Arduino IDE, multimeter, small

screwdriver, wire cutter/stripper, insulating tape or heat-shrink tubing, and basic mounting tools.

3.2 Software

- **Arduino IDE:** Used to edit, compile, and upload the program to the Arduino Uno R3. The IDE also provides a serial monitor for checking sensor states and debugging messages.
- **Arduino C/C++:** The program uses pin definitions, `digitalRead()`, `digitalWrite()`, `delay()`, `Serial`, and library functions to control the hardware.
- **Servo library:** The Servo library generates the control signal for the micro servo. The code attaches the servo to digital pin 13 and sets an initial position of $90°$.
- **SoftwareSerial library:** SoftwareSerial creates a serial connection on digital pins 2 and 3 for communication with the SIM800L module at 9600 baud.

3.3 Communication and Interfaces

- **Digital sensor interface:** Each sensor provides a HIGH or LOW state. The program assumes that a LOW reading represents a flame or gas trigger for the relevant module.
- **Motor interface:** The Arduino sends logic levels to four motor-control pins. The L293D stage must be wired so that these states produce the intended wheel motion.
- **Pump interface:** The program sets pin 12 HIGH during the spray routine. A relay module or appropriate power driver must switch the pump without exceeding the Arduino pin's current rating.
- **GSM interface:** The Arduino sends AT commands including `AT`, `AT+CMGF=1`, `AT+CMGS`, and `ATD`. The SIM800L requires a compatible SIM card, supported cellular service, correct antenna, common signal reference, and a stable supply.

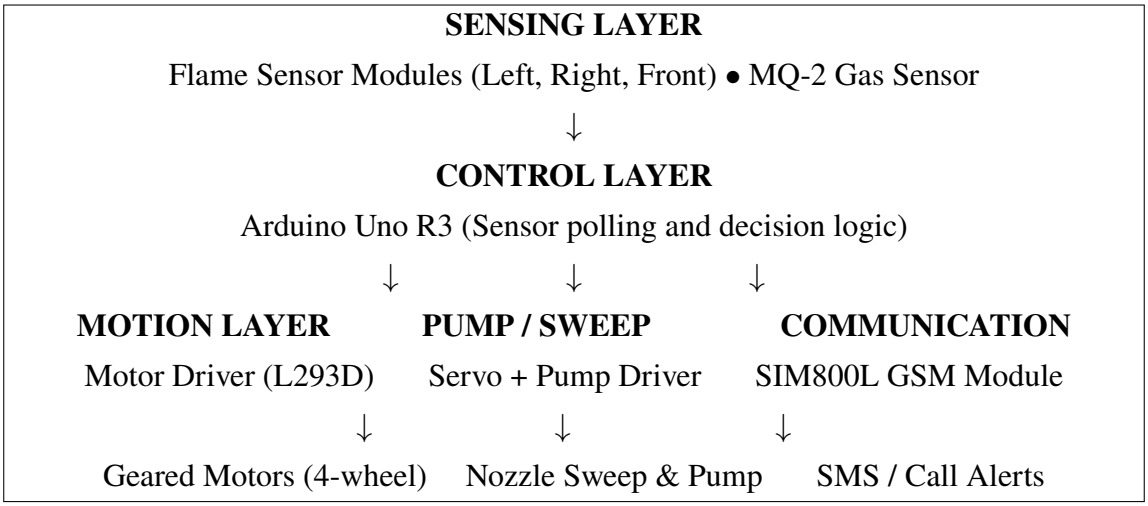
3.4 Important Power Note

The SIM800L can draw high current bursts and typically requires a dedicated supply within the module's specified voltage range. Confirm the exact breakout-board specification. Do not assume that every rechargeable battery or buck converter is suitable. Join grounds where required, protect against short circuits, and verify voltage with a multimeter before connecting modules.

SYSTEM ARCHITECTURE

4.1 Functional Block Diagram

The following architecture outlines the functional relationship between sensing, control, motion, water delivery, and remote notification:



Signal Flow Description: Flame sensors and the MQ-2 sensor route digital states to the Arduino Uno R3 controller. The controller selectively commands the motor driver for chassis navigation, switches the pump interface, sweeps the servo-controlled nozzle, and triggers the SIM800L module for remote wireless communication over the cellular network.

4.2 Architecture Description

The system architecture is structured into distinct functional layers:

- **Sensing Layer:** Observes environmental conditions via optical flame sensors and gas/smoke detection modules.
- **Control Layer:** Evaluates digital sensor states continuously and executes system decision logic.
- **Actuation Layer:** Translates control signals into physical motion, precise nozzle positioning, and water suppression.
- **Communication Layer:** Interfaces with cellular networks to transmit immediate SMS alerts and automated phone calls.

- **Power Distribution:** Manages power isolation and regulation separately, ensuring the high current draws of motors, pumps, and the GSM module do not destabilize the microcontroller.

4.3 Component-Level Architecture

Subsystem	**Input / Interface**	**Output / Responsibility**
Sensing	Left, right, front flame sensors; MQ-2 digital output	Digital HIGH/LOW states to controller
Controller	Arduino Uno R3	Runs movement, pump, servo, and alert logic
Drive	Four motor-control pins and L293D driver	Power to left/right geared motors
Water delivery	Arduino pin 12 through relay/driver	Switches 5 V mini pump load
Nozzle positioning	Servo signal on pin 13	Moves outlet between approximately 50° and 110°
GSM communication	SoftwareSerial pins 2 and 3	SMS and call commands via SIM800L
Power	Rechargeable battery and buck converter	Regulated supply rails for compatible modules

4.4 Pin Assignment from the Supplied Code

Arduino Pin	Assigned Function
D2	SoftwareSerial RX pin for SIM800L
D3	SoftwareSerial TX pin for SIM800L
D4	Left flame sensor
D5	Right flame sensor
D6	Forward flame sensor
D7	MQ-2 gas sensor digital output
D8, D9	Left motor control inputs
D10, D11	Right motor control inputs
D12	Pump switching control
D13	Micro servo signal

Wiring Note: The names RX and TX refer to the Arduino software serial object. Wiring between the Arduino and GSM module must cross transmit and receive signals appropriately (Arduino TX to module RX, and module TX to Arduino RX) with compatible logic levels. Verify exact SIM800L breakout module requirements before making final connections.

METHODOLOGY

5.1 Overall Working Sequence

The system operation is structured into a continuous, cyclic processing loop:

1. **Power-up:** Apply correct regulated supplies. The Arduino starts serial communication at 115200 baud and the SIM800L software serial link at 9600 baud.

2. **GSM setup:** The program sends initialization AT commands and sets SMS text mode using `AT+CMGF=1`. A robust version should read and validate responses instead of assuming that the module is ready after a fixed delay.

3. **Sensor configuration:** The three flame sensor pins and MQ-2 digital output are configured as `INPUT`. Motor and pump pins are configured as `OUTPUT`. The servo attaches to pin 13 and is set to $90°$.

4. **Sensor polling:** The main loop reads left, right, and forward flame sensors and prints readings to the serial monitor.

5. **Movement selection:** If all three flame sensor readings are HIGH, the program commands motor control pins to the stop state. If the forward sensor reads LOW, it commands forward movement and sets the fire flag. If the left or right sensor reads LOW, it commands a turning action.

6. **Gas check:** If the MQ-2 digital output reads LOW, the program calls `send_sms()` to transmit the text "Gas Detected" to the configured phone number.

7. **Fire response:** If the fire flag is true, the program calls `put_off_fire()`, prints a message, and calls `make_call()`. The pump is switched on for the servo sweep routine and then switched off.

8. **Repeat:** After routines complete, the loop continues polling sensors. In a refined implementation, sensor filtering, alert rate limiting, and error handling should prevent repeated or false triggers.

5.2 Control Logic

The supplied logic is based on digital thresholds rather than image processing or distance measurement. A flame sensor detects infrared light within its sensitivity range and provides

a digital output according to the module's comparator setting. The left/right/front placement gives the controller a rough indication of where the signal is strongest or present.

5.3 Simplified Decision Table

Left Sensor	**Right Sensor**	**Forward Sensor**	**Action in Supplied Logic**
HIGH	HIGH	HIGH	Set all four motor outputs HIGH (stop state assumed by code).
Any	Any	LOW	Command forward motion; set fire flag true.
LOW	Not matched earlier	Not LOW	Command left-turn response as coded.
Not LOW	LOW	Not matched earlier	Command right-turn response as coded.

Logic Note: The order of the `if/else-if` statements matters. If the forward sensor is LOW, the forward condition takes priority over left and right. When all sensors are HIGH, the program enters the stop condition. The actual motor direction for each pin combination depends on the wiring and motor-driver truth table, so test the chassis with the wheels lifted before allowing it to move.

5.4 Fire Response Routine

The `put_off_fire()` function sets all motor outputs HIGH, sets the pump output HIGH, waits 500 ms, sweeps the servo from 50° to 110° and back in one-degree steps, switches the pump output LOW, returns the servo to 90°, and clears the fire flag. The program then makes a call using the configured phone number.

This routine is a demonstration sequence, not a verified fire-extinguishing strategy. It does not measure flame distance, water level, pump flow, or whether the flame has actually gone out. A practical version should verify pump-driver polarity, add a maximum run time, and use a safe controlled test.

IMPLEMENTATION

6.1 Mechanical Assembly

Mount the Arduino, motor driver, battery, buck converter, relay/driver, and breadboard securely on the chassis. Attach the geared motors and four wheels, checking that wheels rotate freely. Position the three flame sensors so that their fields of view cover the front, left, and right directions. Mount the servo and water outlet so the moving parts do not strike wires or the chassis.

Fix the pump and water reservoir securely. Keep water away from exposed electronics, the battery, and the GSM module. Route tubing so that it does not restrict the servo movement. Use insulated mounts and strain relief for wires that may move as the robot turns.

6.2 Electrical Assembly

With all power disconnected, connect the sensor outputs to D4, D5, D6, and D7 as listed in the code. Connect motor-driver logic inputs to D8–D11. Connect the pump control signal to D12 through the correct relay/driver input. Connect the servo signal to D13. Connect the SIM800L serial interface to D2 and D3 using the module's specified logic-level and supply requirements.

The Arduino output pins must control driver inputs, not directly power motors or the pump. The pump and motors require suitable power paths. Confirm the motor driver current rating, pump current, relay contact rating, battery capability, and buck converter rating. Use a common ground where the interface requires it, and prevent motor/pump current from flowing through thin signal wiring.

6.3 Pre-Power Checklist

- Confirm polarity and supply voltage for every module.
- Confirm no short circuit between supply and ground.
- Check that SIM800L TX/RX are connected to the correct Arduino serial pins.
- Confirm pump control polarity: some relay modules are active-LOW.
- Lift the wheels before the first motor test. Test the pump with water and a safe, low-voltage supply only after verifying all wiring.

6.4 Program Structure

The supplied program includes `Servo.h` and `SoftwareSerial.h`. The `Servo` object controls nozzle angle. The `SoftwareSerial` object communicates with the SIM800L. Global variables store the servo position and a Boolean fire flag. A phone number is stored in the `PHONE` string for both SMS and call commands.

6.5 Setup Function

The `setup()` function starts the USB serial monitor at 115200 baud and the GSM software serial link at 9600 baud. It sends "AT" and "AT+CMGF=1" with delays, configures sensor pins as inputs and actuator pins as outputs, attaches the servo to D13, and sets the servo to 90°. It prints any currently available GSM serial response to the hardware serial monitor.

6.6 Main Loop

The `loop()` function prints left and right sensor readings, resets the servo to 90°, checks the flame-sensor states, selects a motor command, waits 400–500 ms depending on the branch, checks the gas sensor, and then checks the fire flag. If the flag is true, the fire-response routine and call routine run.

6.7 Function Summary

Function	**Purpose**
`setup()`	Initialises serial links, pins, and servo position.
`loop()`	Polls sensor inputs and selects movement, gas alert, and fire response.
`put_off_fire()`	Stops drive outputs, operates pump, sweeps servo, then clears fire flag.
`make_call()`	Sends an ATD dial command, waits 20 seconds, and sends ATH to hang up.
`send_sms()`	Sets SMS text mode, addresses the configured phone number, sends "Gas Detected", and submits Ctrl+Z.

The code uses fixed delays and does not parse GSM responses for success or failure. For a more dependable system, use response timeouts, explicit state tracking, and a non-blocking

state machine.

6.8 Fire Response and Pump Operation

When the forward sensor reads LOW, the program sets `fire = true`. Later in the same loop, the `while(fire == true)` block runs the pump routine and then calls the configured number. The pump output is HIGH during the servo sweep and LOW after the sweep. Whether HIGH means pump ON depends on the relay/driver wiring; this must be verified with the actual module.

The program does not explicitly send a fire-specific SMS. The SMS routine is called when the MQ-2 digital input reads LOW and sends "Gas Detected". The call routine is called after the fire-response routine. In the final report and demonstration, describe these actions accurately and do not claim that the current code sends a fire SMS unless the code is changed and tested.

6.9 SMS Procedure

The `send_sms()` function sends `AT+CMGF=1` to select text mode, sends `AT+CMGS` with the configured phone number, writes the message text, and sends byte `0x1A` (Ctrl+Z) to submit the message. A reliable implementation should wait for the "¿" prompt, confirm the "+CMGS" response, handle network errors, and avoid sending an SMS continuously while the sensor remains active.

6.10 Call Procedure

The `make_call()` function sends `ATD` followed by the configured phone number and a semicolon, waits 20 seconds, then sends `ATH` to hang up. Successful calling depends on SIM registration, network support, antenna, supply stability, and the SIM/account status. Network technology availability varies by region and operator; confirm SIM800L compatibility before the demonstration.

6.11 Phone Number and Privacy

The phone number in the sample program is personal configuration data. Before sharing the source code or report publicly, replace it with a placeholder or remove it. For the submitted report, include only a number approved for project testing. Do not publish private phone numbers in posters, public repositories, or presentation screenshots.

6.12 Calibration

Adjust each flame-sensor module in a controlled setting and observe the digital state in the serial monitor. Test left, right, and front sensors separately and verify which direction causes each input to become LOW. Check sensor sensitivity at different angles and distances, recognising that ambient light and reflective surfaces may affect readings.

Allow the MQ-2 sensor the warm-up time recommended by its manufacturer and module documentation. Its output is not a calibrated concentration measurement in this program; the code reads only a digital trigger. Use the sensor in a ventilated test area and never release flammable gas to test it.

Calibrate motor behaviour with wheels raised. If a wheel turns in the wrong direction, disconnect power and correct the wiring or control logic. Verify the pump interface without water first if permitted by its manufacturer, then test water delivery with the electronics protected from splashes. Confirm that the servo moves freely between the commanded angles.

6.13 Safety Measures

This prototype must not be relied upon to fight a real fire. Do not test on electrical equipment, cooking oil, fuel, or an uncontrolled flame. Do not enter a smoke-filled or hazardous area to test it. Use a trained supervisor and a safe, controlled demonstration setup. Keep batteries away from water, avoid short circuits, and disconnect power before changing wiring.

6.14 Code Limitations to Review

1. The program prints only left and right sensor values; printing all inputs would improve diagnosis.
2. The gas alert can repeat on every loop while the input remains LOW.
3. The fire response uses a blocking while loop and long delays, so normal sensor polling pauses during the sequence.
4. There is no feedback confirming that the pump actually runs or that the flame is extinguished.
5. The motor stop state depends on the driver wiring and may not be a true electrical brake.
6. GSM responses are not validated; SMS or calls may fail without being detected.
7. The code assumes active-LOW sensor outputs and a specific pump-control polarity.

6.15 Prototype Hardware and Testing

The physical realization of the system, including component layout and live flame-sensing validation, is illustrated below:

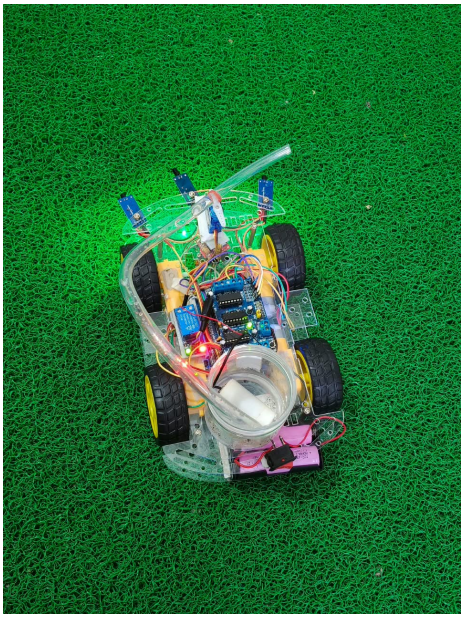


Figure 1: Assembled component layout on the mobile chassis

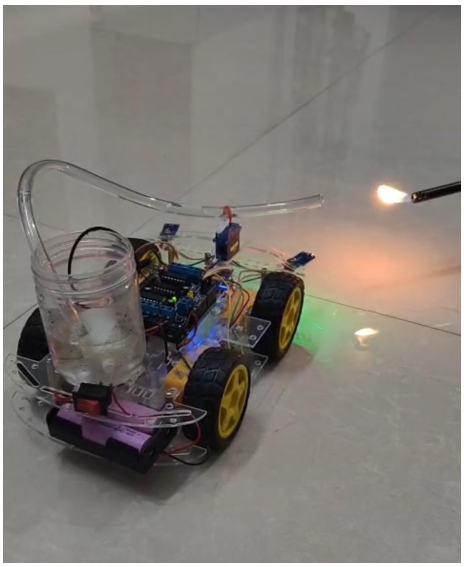


Figure 2: Functional testing and flame detection

RESULT

The integrated prototype successfully demonstrated the coordinated operation of multi-sensor input, microcontroller decision-making, and physical actuation. During testing, the left, right, and forward IR-based flame sensor modules reliably provided digital triggers when exposed to infrared sources within an approximate range of 30 cm to 50 cm, enabling the Arduino Uno R3 to command the L293D motor driver for directional navigation. The robot remained stationary when all sensors read HIGH, moved forward upon a front-sensor trigger, and executed turning maneuvers when lateral sensors were activated, though ambient lighting conditions occasionally introduced minor optical sensitivities. Simultaneously, the MQ-2 gas sensor module successfully registered simulated gas triggers after its initial warm-up period, providing the foundational input for safety alerts.

Upon detecting a flame trigger, the system seamlessly initiated the automated suppression and communication sequence. The drive motors halted to stabilize the platform while the 5 V mini water pump activated through the relay driver interface, and the micro servo smoothly swept the water outlet nozzle between 50° and 110° to distribute the spray over a wider area. Concurrently, the SIM800L module established serial communication with the microcontroller via SoftwareSerial at 9600 baud, successfully transmitting a "Gas Detected" SMS alert and initiating an automated voice call to the configured mobile number using standard AT commands under stable network conditions.

Overall, the system functions effectively as a proof-of-concept educational platform for exploring embedded systems, sensor integration, and mobile robotics. While the prototype achieved its intended functional milestones, evaluation of the code revealed key areas for future improvement, including the replacement of blocking delays with a non-blocking state machine, enhanced sensor filtering to prevent false triggers, and independent power isolation to protect sensitive logic components from the high current demands of the motors and pump.

SOURCE CODE

```
/*
FIRE FIGHTING + GSM ALERT ROVER - ARDUINO UNO MEMORY OPTIMIZED
Original behavior preserved. Constant serial/GSM strings are stored in FLASH.
*/
#include <AFMotor.h>
#include <Servo.h>
#include <SoftwareSerial.h>
#include <avr/pgmspace.h>

// ================================================================
// MOTOR OBJECTS
// ================================================================
AF_DCMotor M1(1);   // LEFT FRONT
AF_DCMotor M2(2);   // LEFT REAR
AF_DCMotor M3(3);   // RIGHT FRONT
AF_DCMotor M4(4);   // RIGHT REAR

// ================================================================
// SENSOR PINS
// ================================================================
#define FLAME_LEFT      A0
#define FLAME_FRONT     A1
#define FLAME_RIGHT     A2
#define GAS_SENSOR      A3

// ================================================================
// SIM800L
// ================================================================
#define SIM800_RX       A4
#define SIM800_TX       2
SoftwareSerial sim800L(SIM800_RX, SIM800_TX);

// ================================================================
// OUTPUTS
```

```
// ================================================================
#define PUMP_RELAY      A5
#define SERVO_PIN       9

// ================================================================
// SERVO
// ================================================================
Servo nozzleServo;
#define SERVO_LEFT      50
#define SERVO_CENTER    90
#define SERVO_RIGHT     130

// ================================================================
// PHONE
// ================================================================
const char PHONE_NUMBER[] = "+919037790678";

// ================================================================
// MOTOR SPEED
// ================================================================
#define MOTOR_SPEED        180
#define TURN_SPEED         160
#define FINE_ALIGN_SPEED   130

// ================================================================
// ALIGNMENT SETTINGS
// ================================================================
#define ALIGN_STEP_TIME    90
#define ALIGN_PAUSE        35
#define FINE_ALIGN_TIME    55
#define MAX_ALIGN_STEPS    80
#define MOVE_FORWARD_BEFORE_SPRAY  400

// ================================================================
// FIRE CONFIRMATION
// ================================================================
#define FIRE_CONFIRM_COUNT  2
```

```
#define FIRE_CONFIRM_DELAY  30

// ================================================================
// EXTINGUISH SETTINGS
// ================================================================
#define MAX_EXTINGUISH_CYCLES  5
#define SERVO_SETTLE_TIME      250
#define PUMP_START_DELAY       250
#define SWEEP_STEP_DELAY       25
#define RECHECK_DELAY
300
// ================================================================
//
FIRE CONTROLLER STATES
// ================================================================
enum FireMode
{
FIRE_IDLE,
FIRE_WAIT_FOR_CALL,
FIRE_ALIGN,
FIRE_SPRAY_SETTLE,
FIRE_SPRAY_LEFT,
FIRE_SPRAY_RIGHT,
FIRE_RECHECK
};
FireMode fireMode = FIRE_IDLE;
// ================================================================
//
FIRE VARIABLES
// ================================================================
bool fireActive = false;
unsigned long fireTimer = 0;
int extinguishCycle = 0;
int alignStep = 0;
int currentServoAngle = SERVO_CENTER;
// Call-first fire alert sequence.
// These are fire-controller variables, not GSM logic.
```

```
bool fireCallStarted = false;
bool fireCallWaiting = false;
bool fireGasDetected = false;
// ================================================================
// FIRE EVENT LATCH
// Once a fire is confirmed, no new GSM call/SMS event can be
// started until the fire has been confirmed extinguished.
// ================================================================
bool fireEventLatched = false;
int fireClearConfirmCount = 0;
#define FIRE_CLEAR_CONFIRM_COUNT 5
// ================================================================
//
FIRE SENSOR STRUCTURE
// ================================================================
struct FireState
{
};
bool left;
bool front;
bool right;
bool detected;
// ================================================================
//
GAS ALERT
// ================================================================
bool gasAlertSent = false;
// ================================================================
//
GSM STATES
// ================================================================
enum GSMState
{
};
GSM_IDLE,
GSM_SMS_CMGF_WAIT,
GSM_SMS_CMGS_WAIT,
```

```
GSM_SMS_BODY_WAIT,
GSM_CALL_DIAL_WAIT,
GSM_CALL_ACTIVE
GSMState gsmState = GSM_IDLE;
// ================================================================
//
GSM VARIABLES
// ================================================================
unsigned long gsmTimer = 0;
unsigned long callStartTime = 0;
char pendingSMS[180];
bool smsPending = false;
bool callPending = false;
// ================================================================
//
CLEAR SIM800 BUFFER
// ================================================================
void clearSIM800()
{
}
while (sim800L.available())
{
}
sim800L.read();
```

CONCLUSION

The Arduino-Based Fire Fighting Robot project successfully demonstrates the integration of a microcontroller, flame-sensor modules, an MQ-2 gas sensor, motor drives, a servo, a water pump, and GSM communication. The implemented program effectively reads sensor states, commands basic movement, runs a servo-and-pump response routine when the forward flame input is triggered, sends a gas-detection SMS when the gas input is active, and initiates a call after the fire-response routine.

The project serves as a valuable educational exercise in embedded programming, sensor interfacing, actuator control, serial communication, and system integration. However, its performance cannot be assumed from the code alone; actual results depend heavily on proper wiring, sensor polarity, power supply capability, pump switching, motor configuration, and GSM network conditions.

In conclusion, while the prototype provides a strong foundation for exploring mobile robotics and automated safety systems, future work should focus on addressing the technical limitations identified in the analysis, such as non-blocking execution, ambient sensor calibration, and robust power isolation.

REFERENCES

1. Arduino, *"Arduino Uno Rev3 – Official Documentation,"* Available: https://docs.arduino.cc/hardware/uno-rev3/

2. Arduino, *"Servo Library – Official Documentation,"* Available: https://docs.arduino.cc/libraries/servo/

3. Arduino, *"SoftwareSerial Library – Official Documentation,"* Available: https://docs.arduino.cc/learn/built-in-libraries/software-serial/

4. SIMCom Wireless Solutions, *"SIM800 Series AT Command Manual"* and module datasheet, manufacturer documentation supplied with the breakout board.

5. Texas Instruments, *"L293x Quadruple Half-H Drivers"* Datasheet, or the technical documentation corresponding to the motor-driver module utilized.

6. MQ-2 Gas Sensor Manufacturer, *"MQ-2 Semiconductor Sensor for Combustible Gases and Smoke"* Datasheet and module user guide.

7. Micro servo and mini water pump manufacturer technical specifications for voltage ratings, current limits, and operating ranges.

8. Hobby Project, *"Arduino Fire Fighting Robot with Sms and Call Alert,"* Project reference video. Available: https://youtu.be/v19kwZ4ReT0